

AI技术应用场景匹配报告

文档版本：V1.0

所属项目：基于协同过滤算法的个性化电影推荐系统

撰写人：王雪琳

日期：2026年6月

1. 2024–2026年AI在软件工程领域的应用调研

在写这份报告前，我主要看了 2024 年前后的几份公开资料。个人感觉 AI 不只是帮忙写代码，需求、设计、测试等环节也有人开始用。信通院 2024 年发布的 AI4SE 调查报告里写到，企业在软件工程各阶段尝试 AI 的情况在增多[1]。

下面按软件开发的几个阶段，简单梳理一下我了解到的应用情况。

1.1 需求分析阶段

需求分析阶段比较明显的变化是，AI 可以把比较口语化的描述整理成相对规范的需求文档。

- 需求文档辅助生成**：比如输入“做一个电影推荐功能，新用户没有评分记录时要有默认推荐”，AI 可以帮忙整理成用户故事，并补充一些容易遗漏的情况。
- 需求一致性检查**：如果项目需求比较多，AI 可以帮助对照已有需求，看看有没有前后矛盾的地方。比如“推荐要实时算”和“推荐结果提前算好存数据库”就不能同时成立。

报告里对需求分析阶段的调查，也能看出使用 AI 的企业在增加[1]。

1.2 系统设计阶段

设计阶段 AI 主要帮助的是“出初稿”，最终方案还是要开发者自己判断。

- 架构图和 ER 图辅助绘制**：输入系统描述后，AI 可以生成初版的架构图、数据库 ER 图，开发者再修改。
- 技术方案参考**：对于推荐系统这类项目，AI 可以给出一些常见做法，比如用 Redis 缓存热门电影、用策略模式切换不同推荐算法等，供设计时参考。

1.3 编码实现阶段

这是我感受最深的一个阶段，因为开发项目时确实用到了 Cursor 等工具。

- 代码补全和模块生成**：GitHub Copilot、Cursor 这类工具可以根据项目上下文给出代码建议，对于 Django 视图、前端页面这类重复性工作帮助比较大。
- 代码问题提示**：AI 除了检查语法，有时还能指出明显的性能问题，比如循环里反复查数据库。

同一份报告还提到，部分企业里 AI 生成代码已占一定比重[1]，说明不只是概念讨论。

1.4 测试验证阶段

测试阶段是 AI 应用比较集中的领域之一。中国信通院2024年发布的《智能化软件工程技术和应用要求 第3部分：智能测试能力》，对 AI 测试相关能力做了比较系统的划分，包括测试分析、测试执行、结果评估等方面[2]。

结合资料和我的理解，目前比较常见的做法有：

- **自动生成测试用例**：把函数或视图代码交给 AI，让它帮忙写正常情况、边界情况和异常情况下的测试。
- **测试脚本维护**：页面结构变了以后，部分 AI 测试工具可以尝试自动调整定位方式，减少脚本大面积失效的问题。

1.5 运维监控阶段

运维方面我接触不多，但从资料来看，AI 主要用于：

- **异常排查**：把日志、报错信息汇总后，让 AI 帮忙分析可能原因。
- **资源调度参考**：根据访问高峰规律，辅助判断是否需要提前扩容。

由于我的项目目前还是本地部署，这部分暂时没有实际用到，但可以作为后续扩展方向了解。

2. 场景匹配分析

2.1 结合本项目的背景

我的项目是一个基于协同过滤算法的电影推荐系统，技术栈是 Django + MySQL + Bootstrap，由我一个人开发。在第一次作业里，我把开发过程分成了4个 Sprint：

Sprint	模块	主要功能
Sprint 1	用户模块	注册、登录、登出
Sprint 2	电影展示模块	首页、分类、搜索、热门电影
Sprint 3	评分与推荐模块	评分、评分历史、协同过滤推荐
Sprint 4	AI问答模块	调用 DeepSeek 做电影问答

实际开发过程中，我觉得比较费时、也比较容易出问题的环节主要有四类，下面分别说明为什么适合引入 AI 辅助。

```
graph LR
    A[需求分析] -->|场景一| B[推荐功能需求整理]
    C[编码实现] -->|场景二| D[推荐代码优化]
    C -->|场景四| E[AI问答模块开发]
    F[测试验证] -->|场景三| G[View测试用例编写]
```

场景一：推荐功能需求的整理与补充（需求分析阶段）

对应 Sprint：Sprint 3（评分与推荐模块）

我遇到的问题：

做推荐功能之前，如果只是笼统地写"给用户推荐电影"，后面开发时很容易漏掉一些情况。以我项目里的 RecommendMovieView 为例，实际逻辑并不简单，至少包括：

- 找到和当前用户评分记录相近的其他用户
- 排除用户已经评过的电影
- 按相似用户的评分结果排序后推荐

如果需求阶段没想清楚，后面很容易返工。我一开始就没单独考虑这些情况，比如：

- 新用户没有任何评分时怎么办（冷启动）
- 找不到足够多相似用户时怎么办
- 最后推荐列表为空时页面怎么显示

为什么适合用 AI：

这类问题比较适合先用自己的话描述功能，再让 ChatGPT 或 DeepSeek 帮忙整理成用户故事和验收标准。比如输入：

"帮我写一个电影推荐功能的用户故事，基于用户协同过滤，K=20，N=10，还要考虑新用户没有评分的情况。"

AI 通常会把一些容易忽略的点一起列出来，比如"没有评分时展示热门电影"。我自己写需求时往往会先写主流程，边界情况经常后面才想到。

预期作用： 主要是减少需求遗漏，让开发前先把情况考虑全一些，避免后面改来改去。

场景二：推荐模块代码的性能优化（编码实现阶段）

对应 Sprint：Sprint 3（评分与推荐模块）

我遇到的问题：

推荐功能写在 `movie/views.py` 的 `RecommendMovieView` 里。当前做法是对每个其他用户分别查数据库，统计和当前用户评过的电影有多少重合：

```
for user in other_users:
    user_sim_dct[user.id] = len(Movie.objects.filter(user=user) & self.cur_user_movie_qs)
```

我在本地跑的时候能明显感觉到这个页面比较慢，控制台里也会打印耗时。原因是用户数量一多，这种写法会产生大量数据库查询。其实我在 `tests.py` 里还写过一个 `UserCFRec` 类，用了倒排表和相似度缓存，思路更合理一些，但还没有真正用到 Web 页面里。

为什么适合用 AI：

这类问题很适合拿给 Cursor 或 Copilot 看，让它帮忙分析哪里慢、怎么改。比如：

1. 指出循环里反复查库的问题
2. 建议参考 `tests.py` 里已有的倒排表思路
3. 给出批量查询或预计算相似度的改法

AI 不一定一次改对，但至少能比较快给出几个可行方向，比自己从头查资料要快。

预期作用： 缩短排查和重构时间，提升推荐页面的响应速度。具体能快多少，还要等实际改完后再看。

场景三：View 层测试用例的编写（测试验证阶段）

对应 Sprint： Sprint 3 和 Sprint 4

我遇到的问题：

我现在的 `movie/tests.py` 里，主要是之前做协同过滤算法实验时写的 `UserCFRec` 测试代码，针对网页功能的测试基本没有。如果要自己补测试，需要考虑的点很多，比如：

- `RecommendMovieView`：没登录能不能访问、没有推荐结果时页面是否正常
- `AIChatView`：输入为空、API 报错、网络超时怎么处理
- `MovieDetailView`：重复评分是更新还是新增、没登录能不能提交评分

这些场景如果全部手写 Mock 数据和测试代码，工作量不小，而且很容易漏。

为什么适合用 AI：

可以把 View 源码发给 AI，让它按 Django `TestCase` 的写法先生成一版测试框架。AI 比较擅长把“正常流程 + 几种异常情况”先列出来，我再根据项目实际情况删改。信通院的智能测试标准里也提到，AI 可以在测试设计和测试执行环节提供辅助[2]。

预期作用： 降低测试代码的起步成本，让项目不至于完全没有自动化测试。

场景四：AI 问答模块的开发（编码实现阶段）

对应 Sprint：Sprint 4（AI问答模块）

我遇到的问题：

项目里有一个 AI 问答页面，需要调用 DeepSeek API，并且做成流式输出。这部分对我来说是全新的内容，涉及：

- 后端用 `StreamingHttpResponse` 转发流式数据
- 前端用 JavaScript 解析 SSE 数据并逐字显示
- API Key、超时、报错等细节处理

如果完全自己查文档一点点写，需要先把这些概念都弄明白，学习成本比较高。

为什么适合用 AI：

我实际开发这个功能时，就是用 Cursor 辅助完成的。做法是先描述需求：

"在 Django 里做一个电影问答页面，调用 DeepSeek API，支持流式返回，前端实时显示回复。"

然后 AI 生成了 `AIChatView` 和 `ai_chat.html` 的初版代码，我再自己改 API Key、系统提示词和页面样式。相比完全手写，这种方式让我更快把功能跑起来。

预期作用：降低新功能的上手难度，尤其适合以前没接触过的 API 集成类开发。

3. 四个场景汇总

场景	所属阶段	对应 Sprint	主要问题	打算使用的工具
场景一：推荐需求整理	需求分析	Sprint 3	边界情况容易遗漏	ChatGPT / DeepSeek
场景二：推荐代码优化	编码实现	Sprint 3	推荐页面响应慢	Cursor / Copilot
场景三：View 测试编写	测试验证	Sprint 3/4	缺少网页层测试	Cursor
场景四：AI 问答开发	编码实现	Sprint 4	流式 API 集成不熟悉	Cursor

4. 小结

结合我的项目实际情况，我认为 AI 最适合介入的并不是整个开发流程，而是那些**重复性强、规则多、或者自己不太熟悉**的环节。比如需求补充、测试代码初稿、API 集成等。

这4个场景分别对应第一次作业里 Sprint 3 和 Sprint 4 的开发内容，覆盖了需求分析、编码实现和测试验证三个阶段。后续我会在过程改进和原型验证中，挑选其中1-2个场景做实际测试，看看 AI 辅助和纯手工相比，到底省了多少时间、质量怎么样。

参考文献

[1] 中国信息通信研究院. AI4SE行业现状调查报告（2024年度）[R]. 2024.

[2] 中国信息通信研究院. 智能化软件工程技术和应用要求 第3部分：智能测试能力（AIIA/PG 0138-2024）[S]. 2024.